SOFTWARE SYSTEM DOCUMENTATION

Craftalism

Modular Minecraft Economy Platform

Author **Henrique Michelini**

Version **1.0**

Date **2026-03-28**

License **MIT**

github.com/HenriqueMichelini

Contents

Project Introduction 1

1 Craftalism API 3
- 1.1 Overview 3
- 1.2 Responsibilities 3
- 1.3 Architecture and Design 3
  - 1.3.1 Layer Descriptions 3
  - 1.3.2 Security Model 4
  - 1.3.3 Error Contract 4
  - 1.3.4 API Endpoints 4
- 1.4 Tech Stack 5
- 1.5 Configuration 5
- 1.6 Known Limitations 5
- 1.7 Roadmap 5

2 Craftalism Authorization Server 7
- 2.1 Overview 7
- 2.2 Responsibilities 7
- 2.3 Architecture and Design 7
  - 2.3.1 Filter Chain Model 7
  - 2.3.2 Key Configuration 8
  - 2.3.3 Client Provisioning 8
- 2.4 Tech Stack 8
- 2.5 Configuration 9
- 2.6 Known Limitations 9
- 2.7 Roadmap 9

3 Craftalism Economy 10
- 3.1 Overview 10
- 3.2 Responsibilities 10
- 3.3 Architecture and Design 10
  - 3.3.1 Layer Descriptions 10
  - 3.3.2 The `/pay` Transfer Flow 11
  - 3.3.3 Player Join Provisioning 11
- 3.4 Tech Stack 11
- 3.5 Configuration 11
- 3.6 Known Limitations 12
- 3.7 Roadmap 12

4 Craftalism Dashboard 13
- 4.1 Overview 13
- 4.2 Responsibilities 13
- 4.3 Architecture and Design 13
  - 4.3.1 Layer Descriptions 13
  - 4.3.2 Runtime Configuration Injection 14
- 4.4 Tech Stack 14
- 4.5 Configuration 14
- 4.6 Known Limitations 14
- 4.7 Roadmap 14

5 Craftalism Deployment 16
- 5.1 Overview 16
- 5.2 Responsibilities 16

5.3 Architecture and Design 16
5.3.1 Service Topology 16
5.3.2 Startup Dependency Order 16
5.3.3 Port Mapping 17
5.4 Tech Stack 17
5.5 Configuration 17
5.6 Known Limitations 17
5.7 Roadmap 18

A Shared Environment Variables 19

B Quick-Start Command Reference 20

Project Introduction

What Craftalism Is

Craftalism is a modular, production-oriented economy platform that integrates a Minecraft game server with a fully externalized web backend. Rather than storing economy state inside the game server — the conventional approach for Minecraft plugins — the platform delegates all balance and transaction data to a centralized REST API. The Minecraft plugin becomes a thin, authenticated client: it handles commands and player interaction, then delegates all state reads and writes to the backend over HTTP.

This architecture allows the economy to be administered, audited, and observed entirely outside the game, through a web dashboard, while the game server itself remains stateless with respect to financial data.

System Architecture

The platform is composed of five independent services, each in its own repository. Service-to-service communication uses HTTP; the Minecraft plugin authenticates to the Craftalism API with OAuth2 bearer tokens, while the Dashboard currently calls the API without authentication.

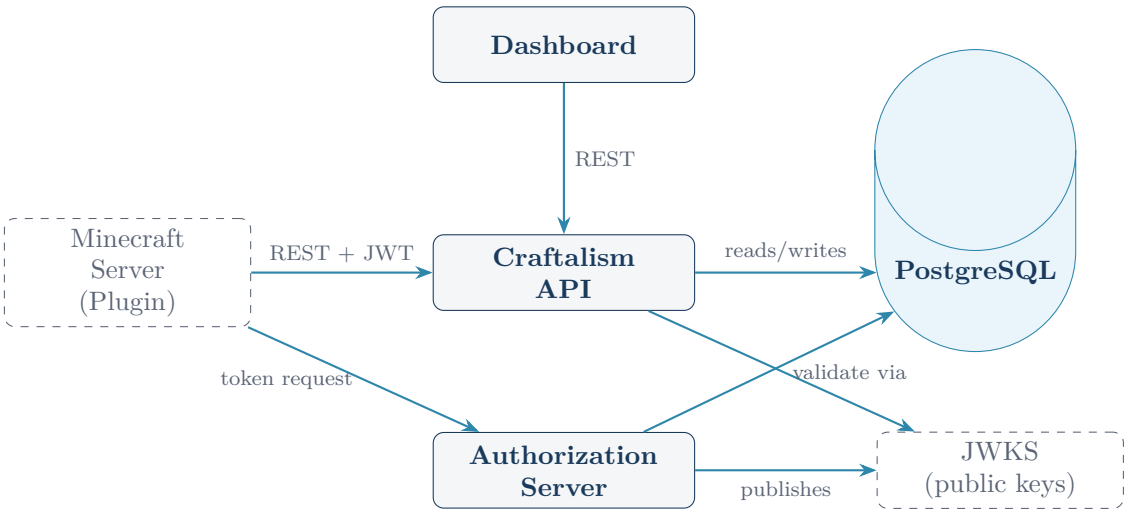

Figure 1: Craftalism system topology

1. The Minecraft plugin authenticates with the **Authorization Server** on startup using the `client_credentials` OAuth2 grant, and caches the resulting JWT access token.
2. All subsequent plugin calls to the **Craftalism API** carry that JWT as a `Bearer` token in the `Authorization` header.
3. The API validates tokens *locally* by fetching the Authorization Server's public keys from `/oauth2/jwks` — no per-request round-trip to the auth server is required.
4. The **Dashboard** calls the same API through an Nginx reverse proxy and renders the data for administrators; these requests are currently unauthenticated.
5. Both the API and the Authorization Server persist state to a shared **PostgreSQL** instance,

in separate databases (`craftalism` and `authserver`).

Repository Index

Repository	Primary Stack	Purpose
`craftalism-api`	Java 17, Spring Boot 3.5, JPA	Core REST API for players, balances, and transactions.
`craftalism-authorization-se`	Java 17, Spring Authorization Server	OAuth2/OIDC token issuer for internal services.
`craftalism-economy`	Java 21, Paper API 1.21.4	Minecraft plugin; delegates all economy state to the API.
`craftalism-dashboard`	React 19, TypeScript 5, Vite 7	Admin web UI for read-oriented data visibility.
`craftalism-deployment`	Docker Compose, PostgreSQL 18	Container orchestration for the full platform.

Shared Conventions

The following conventions apply across all modules and are referenced throughout this document.

- **Authentication:** The Minecraft plugin authenticates with short-lived JWT access tokens issued via the `client_credentials` grant. The Craftalism API validates these tokens locally using the published JWKS.
- **Error responses:** The API returns RFC 9457 `ProblemDetail` payloads for all error conditions, with a consistent set of extension fields: `type`, `detail`, `status`, `timestamp`, `path`, and `errors`.
- **Scope enforcement:** Read operations require the `api:read` scope. Write operations require `api:write`.
- **Containerization:** All services are distributed as Docker images and orchestrated with Docker Compose via the `craftalism-deployment` repository.

Module 1

Craftalism API

Java 17 · Spring Boot 3.5 · PostgreSQL

Core REST API — player registration, balance management, and transaction records

1.1 Overview

The Craftalism API is the central data service for the economy platform. It exposes a typed REST interface under `/api` that is consumed by both the Minecraft plugin and the Dashboard. All state for players, balances, and transactions is owned and persisted by this service. Write operations are protected by JWT scope enforcement; the API trusts tokens issued by the Craftalism Authorization Server.

1.2 Responsibilities

- Registers new players and resolves existing players by UUID or display name.
- Creates, reads, and overwrites balance records for registered players.
- Processes deposit and withdrawal operations on individual balances.
- Stores transaction records between players as an audit log.
- Enforces OAuth2 scope rules: `api:read` for all `GET` requests, `api:write` for all mutating requests.
- Returns standardized RFC 9457 `ProblemDetail` payloads for all error conditions.
- Exposes interactive API documentation via Swagger UI on the local profile.

1.3 Architecture and Design

The codebase follows a classic layered architecture. Each layer communicates only with the layer directly below it; no layer skips levels or introduces circular dependencies.

1.3.1 Layer Descriptions

Controller layer (`controller/`): Exposes REST endpoints under `/api/**`. Validates request DTOs using Bean Validation annotations and returns typed DTO responses with appropriate HTTP status codes. Contains no business logic.

Service layer (`service/`): Encapsulates business rules and transactional behavior. Throws typed domain exceptions (e.g., insufficient funds, duplicate player) that the exception handler maps to `ProblemDetail` responses.

Repository layer (`repository/`): Spring Data JPA repositories with custom queries for top-balance ranking and pessimistic locking on concurrent balance updates.

Domain/model layer (`model/`): JPA entities — `Player`, `Balance`, `Transaction`. Contains no behavior beyond JPA annotations.

Cross-cutting concerns: `config/` holds the security and OpenAPI configuration; `exceptions/`

holds the centralized exception handler; `mapper/` holds entity-to-DTO mapping components.

1.3.2 Security Model

The API is a stateless OAuth2 resource server. JWTs are validated locally against the issuer URI configured in `AUTH_ISSUER_URI`. No session is created.

Scope	Permitted operations
`SCOPE_api:read`	`GET /api/**`
`SCOPE_api:write`	`POST`, `PUT`, `PATCH`, `DELETE` on `/api/**`
Public (no token required): `GET /actuator/health, /swagger-ui/**, /v3/api-docs/**`	

1.3.3 Error Contract

All errors are returned as RFC 9457 `ProblemDetail` with the following extension fields:

Field	Description
`type`	URI identifying the error category: `.../validation`, `.../business-rule`, or `.../internal`.
`detail`	Human-readable explanation of the specific error.
`status`	HTTP status code as an integer.
`timestamp`	ISO 8601 timestamp of when the error occurred.
`path`	The request path that produced the error.
`errors`	Field-level validation map (present on `.../validation` errors only).

1.3.4 API Endpoints

Method	Path	Description
Players		
GET	`/api/players`	List all players.
GET	`/api/players/{uuid}`	Get player by UUID.
GET	`/api/players/name/{name}`	Get player by display name.
POST	`/api/players`	Register a new player.
Balances		
GET	`/api/balances`	List all balances.
GET	`/api/balances/{uuid}`	Get balance by player UUID.
GET	`/api/balances/top`	Top balances (`?limit=`, clamped 1–20, default 10).
POST	`/api/balances`	Create a balance record.
PUT	`/api/balances/{uuid}/set`	Overwrite a balance amount.
POST	`/api/balances/{uuid}/deposit`	Deposit funds.
POST	`/api/balances/{uuid}/withdraw`	Withdraw funds.
Transactions		
GET	`/api/transactions`	List all transactions.
GET	`/api/transactions/id/{id}`	Get transaction by ID.
GET	`/api/transactions/from/{uuid}`	List outgoing transactions for a player.
GET	`/api/transactions/to/{uuid}`	List incoming transactions for a player.
POST	`/api/transactions`	Store a transaction record.

> **Important:** `POST /api/transactions` stores a record only. It does **not** call balance transfer logic. Balance state and the transaction ledger are not updated atomically. See Known Limitations.

1.4 Tech Stack

Category	Technology	Notes
Language	Java 17	
Framework	Spring Boot 3.5	Web, Validation, Actuator
Security	Spring Security, OAuth2 Resource Server	Stateless JWT validation
Persistence	Spring Data JPA	Hibernate provider
Migrations	Flyway	Docker profile only
Database	PostgreSQL (Docker profile)	
Database	H2 in-memory (local profile)	Schema recreated on each startup
API Docs	springdoc-openapi	Swagger UI at `/api-docs`
Testing	JUnit 5, Mockito, Spring Test	
Build Tool	Gradle Wrapper	
Packaging	Docker multi-stage image	

1.5 Configuration

Variable	Default	Required	Description
`SPRING_PROFILES_ACTIVE`	—	**Yes**	Set to `local` (H2) or `docker` (PostgreSQL).
`AUTH_ISSUER_URI`	`http://local`	No	JWT issuer URI. Must match the Authorization Server's configured issuer.
`SPRING_DATASOURCE_URL`	—	Docker only	JDBC connection string.
`SPRING_DATASOURCE_USERNAME`	—	Docker only	Database username.
`SPRING_DATASOURCE_PASSWORD`	—	Docker only	Database password.

1.6 Known Limitations

- `POST /api/transactions` does not atomically update balances. The ledger and balance state can diverge if the system fails between a balance operation and the subsequent transaction record write.
- List endpoints return all records in a single response with no pagination or filtering support. This will degrade under large datasets.
- Integration tests run against H2 in-memory, not a real PostgreSQL instance. Schema and query compatibility gaps may exist.
- CORS allowed origins are hard-coded in `SecurityConfig` (localhost ports only), so non-local Dashboard or operator UIs require code changes and rebuilds.
- The repository workflow publishes Docker images on version tags but does not execute automated tests before image publication.

1.7 Roadmap

- Integrate `POST /api/transactions` with `BalanceService.transfer()` so ledger and balance updates occur atomically within a single database transaction.
- Add cursor-based or offset pagination and server-side filtering to all list endpoints.
- Add integration tests against real PostgreSQL with actual JWT-bearing requests.

- Externalize allowed origins into environment variables or profile-specific properties.
- Extend the GitHub Actions workflow to run tests before publishing images.

Module 2

Craftalism Authorization Server

Java 17 · Spring Authorization Server · OAuth 2.1 / OIDC

OAuth 2.1 / OIDC token issuer — machine-to-machine authentication for internal services

2.1 Overview

The Authorization Server is the authentication anchor of the Craftalism platform. It issues short-lived, RSA-signed JWT access tokens to trusted internal clients using the OAuth 2.0 `client_credentials` grant. Downstream services (primarily the Craftalism API) validate these tokens locally by fetching the server's public keys from its JWKS endpoint — no validation round-trip is required per request. The Authorization Server's only registered client is the Minecraft plugin.

2.2 Responsibilities

- Issues RSA-signed JWT access tokens via `POST /oauth2/token` using the `client_credentials` grant.
- Publishes RSA public keys at `GET /oauth2/jwks` for local token verification by downstream services.
- Exposes OIDC discovery metadata at `GET /.well-known/openid-configuration`.
- Exposes OAuth 2.0 server metadata at `GET /.well-known/oauth-authorization-server`.
- Supports token introspection (`/oauth2/introspect`) and revocation (`/oauth2/revoke`).
- Seeds a single registered OAuth client (`minecraft-server` by default) idempotently on every startup.
- Enforces a deny-by-default HTTP policy: all paths not explicitly listed as public return 401/403.

2.3 Architecture and Design

The service is a Spring Boot application with two distinct security filter chains and a startup provisioning routine.

2.3.1 Filter Chain Model

Two filter chains run in priority order:

Chain 1 — Protocol chain (Order 1): Handles all `/oauth2/**` and discovery endpoints according to OAuth 2.0/OIDC protocol rules. Managed entirely by Spring Authorization Server.

Chain 2 — Fallback chain (Order 2): Stateless and session-less. Permits only the paths listed below; all other requests are denied with `401` or `403`.

Permitted path	Reason
`GET /actuator/health`	Health check for orchestrators.
`GET /oauth2/jwks`	Public key material for JWT verification.
`GET /.well-known/oauth-authorization-server`	OAuth 2.0 discovery.
`GET /.well-known/openid-configuration`	OIDC discovery.

2.3.2 Key Configuration

RSA keys are loaded from environment variables (`RSA_PRIVATE_KEY` / `RSA_PUBLIC_KEY`) at startup by `RsaKeyConfig`. If neither variable is set, the service generates an ephemeral key pair.

> **Important:** Ephemeral keys are destroyed on service restart. Any JWT issued with an ephemeral key becomes unverifiable after a restart. Never use ephemeral keys in persistent environments.

2.3.3 Client Provisioning

`ClientRegistrationService` seeds one OAuth client (the Minecraft plugin) on every startup. The operation is idempotent: if the client already exists in the database, no change is made. Client credentials, scopes, and grant types are driven by environment variables.

2.4 Tech Stack

Category	Technology	Notes
Language	Java 17	
Framework	Spring Boot 3.5	
Security	Spring Security, Spring Authorization Server	
Database Access	Spring JDBC	JDBC repositories for clients/authorizations
Database (runtime)	PostgreSQL	
Database (tests)	H2 (PostgreSQL compat. mode)	
Build Tool	Gradle Wrapper	
Packaging	Docker multi-stage image	
Testing	JUnit 5, Spring Test	

2.5 Configuration

Variable	Default	Required	Description
DB_URL	—	**Yes**	JDBC connection string.
DB_USER	—	**Yes**	Database username.
DB_PASSWORD	—	**Yes**	Database password.
MINECRAFT_CLIENT_SECRET	—	**Yes**	Secret for the seeded OAuth client.
AUTH_ISSUER_URI	http://localhos	No	JWT issuer URI. Must match downstream configuration.
MINECRAFT_CLIENT_ID	minecraft-serve	No	Client ID for the seeded OAuth client.
RSA_PRIVATE_KEY	—	No	PEM RSA private key (literal \n separators). Ephemeral fallback if absent.
RSA_PUBLIC_KEY	—	No	PEM RSA public key (literal \n separators). Ephemeral fallback if absent.

2.6 Known Limitations

- No `generate-keys.sh` script exists in this repository, despite being referenced in documentation and the deployment repository. RSA key pairs must be generated manually.
- Integration tests run against H2 in PostgreSQL compatibility mode, not real PostgreSQL. Schema differences may produce false passes.
- `SecurityConfig` includes public matchers for `/api/**` paths that do not belong to this service and appear to be leftover rules, increasing configuration noise and maintenance risk.
- Token introspection and revocation endpoints have no documented usage examples.
- The repository workflow builds and pushes images on tags but does not run automated tests before publication.

2.7 Roadmap

- Add a `generate-keys.sh` utility script for RSA key pair generation and formatting.
- Add containerized integration tests that run against real PostgreSQL.
- Remove non-auth API path matchers from `SecurityConfig` and simplify the fallback chain to auth-server endpoints only.
- Document introspection and revocation usage with curl examples.
- Extend the GitHub Actions workflow to run tests before publishing images.

Module 3

Craftalism Economy

Java 21 · Paper API 1.21.4 · Caffeine

Minecraft plugin — in-game economy commands backed by the Craftalism API

3.1 Overview

The Economy plugin runs inside the Minecraft server and serves as the player-facing interface to the Craftalism economy. Rather than maintaining balance state locally, it acts as an authenticated HTTP client to the Craftalism API. All command handling is fully asynchronous, ensuring the main server thread is never blocked by network calls. An in-process Caffeine cache reduces redundant API requests for player and balance lookups.

3.2 Responsibilities

- Exposes in-game economy commands: `/pay`, `/balance`, `/setbalance`, and `/baltop`.
- Executes all API calls asynchronously using `CompletableFuture` to avoid blocking the main server thread.
- Authenticates with the Authorization Server using `client_credentials` and caches the resulting JWT.
- Caches player and balance records locally using Caffeine to minimize round-trip latency.
- Implements compensating rollback for `/pay`: if a deposit to the receiver fails after a successful withdrawal from the sender, the plugin attempts to reverse the withdrawal.
- Provisions new players and balance records automatically on player join if they do not yet exist in the API.
- Formats currency amounts according to configurable locale, symbol, and fallback rules.

3.3 Architecture and Design

The codebase uses a strict four-layer architecture. Dependencies point inward; no outer layer is visible to an inner one.

3.3.1 Layer Descriptions

Presentation layer (`presentation/`): Bukkit command executors and event listeners. Validates input, delegates to application services, and maps result status codes to player-facing messages sourced from `logs.yml`.

Application layer (`application/`): Use-case orchestration. Each service coordinates domain rules and infrastructure calls, returning typed result DTOs. No raw exceptions propagate to the presentation layer.

Domain layer (`domain/`): Core models (`Player`, `Balance`, `Transaction`) and business rules (`FundsTransfer`, `AmountCheck`, `CurrencyFormatter`, `CurrencyParser`). No framework dependencies.

Infrastructure layer (`infra/`): HTTP client, OAuth2 token service, API service classes, Caffeine cache repositories, config loading, and the plugin bootstrap container.

3.3.2 The `/pay` Transfer Flow

1. Validate sender permissions and command arguments.
2. Resolve sender from cache or API; resolve receiver by name from the API.
3. Validate amount (positive, non-zero) and reject self-payment.
4. Withdraw from sender via `POST /api/balances/{uuid}/withdraw`.
5. Deposit to receiver via `POST /api/balances/{uuid}/deposit`.
6. If deposit fails: attempt compensating deposit back to sender (best effort).
7. Write transaction record via `POST /api/transactions` (best effort — failure does not revert a completed transfer).
8. Map final result status to player-facing message.

3.3.3 Player Join Provisioning

On `PlayerJoinEvent`, the plugin calls `GET /api/players/{uuid}`. If the player does not exist, it calls `POST /api/players` to register them and then `POST /api/balances` to create a default balance. Both the player and balance records are then stored in the Caffeine cache.

3.4 Tech Stack

Category	Technology	Notes
Language	Java 21	
Minecraft API	Paper API 1.21.4-R0.1-SNAPSHOT	Paper-compatible servers only
HTTP Client	Java `HttpClient`	Built-in; no additional dependency
Cache	Caffeine	In-process player and balance cache
Serialization	Gson	JSON parsing for API responses
Build Tool	Gradle Wrapper	
Testing	JUnit 5, Mockito, MockBukkit	

3.5 Configuration

`config.yml` — bundled in JAR, written to plugin data folder on first run.

Key	Default	Required	Description
`default-balance`	`100000000`	No	Initial balance (internal scaled integer).
`locale`	`en-US`	No	Locale for currency formatting.
`currency-symbol`	`$`	No	Symbol prepended to formatted amounts.
`null-representation`	—	No	String shown when a value cannot be formatted.

Environment variable overrides (`connection-config.yml` values):

Variable	Overrides
`CRAFTALISM_API_URL`	API base URL.
`AUTH_ISSUER_URI`	OAuth2 issuer URI.
`AUTH_TOKEN_PATH`	Token endpoint path.
`MINECRAFT_CLIENT_ID`	OAuth2 client ID.
`MINECRAFT_CLIENT_SECRET`	OAuth2 client secret. Prefer this over hardcoding in YAML.
`CRAFTALISM_API_KEY`	Fallback for `MINECRAFT_CLIENT_SECRET`.
`MINECRAFT_CLIENT_SCOPES`	Requested OAuth2 scopes (space-separated).

3.6 Known Limitations

- The `OnQuit` listener class exists but is not registered in `EventRegistrar`. Player disconnect events are not handled and the cache is never evicted on logout.
- `BootContainer#shutdown()` is a placeholder and performs no resource cleanup.
- `/setbalance` input validation accepts numeric digit strings only before scaling; decimal input is rejected inconsistently with other commands.
- No integration or end-to-end tests run against a live API instance.
- The release workflow builds with `-x test`, so plugin releases are currently published without automated test execution.

3.7 Roadmap

- Register `OnQuit` in `EventRegistrar` and implement cache eviction on player disconnect.
- Implement `BootContainer#shutdown()` to release HTTP client and cache resources cleanly.
- Fix `/setbalance` input validation to accept decimal amounts consistently with `/pay`.
- Add integration tests against a live Craftalism API instance.
- Update the release workflow to execute the test suite before packaging and publishing artifacts.

Module 4

Craftalism Dashboard

TypeScript 5 · React 19 · Tailwind CSS 3

Admin web UI — operational visibility into players, balances, and transactions

4.1 Overview

The Dashboard is a read-oriented single-page application that lets administrators inspect the current state of the economy. It connects to the Craftalism API through an Nginx reverse proxy and presents player records, balance amounts, and transaction history in a consistent table-based interface. The frontend has no direct database access and no authentication layer of its own.

4.2 Responsibilities

- Renders tabular views of all players, balances, and transactions sourced from the Craftalism API.
- Displays loading, empty, and error states for every data view.
- Provides a retry action on failed API requests without requiring a full page reload.
- Formats dates and currency amounts consistently across all views using centralized utility functions.
- Proxies all `/api/*` requests to the Craftalism API through Nginx to avoid browser CORS restrictions.

4.3 Architecture and Design

The application uses a feature-first structure. Each dashboard section owns its view component and table column configuration. Shared concerns — the API client, data-fetching hook, and formatting utilities — live in dedicated cross-cutting modules.

4.3.1 Layer Descriptions

Feature view layer (`pages/Dashboard/views/`): `PlayersView`, `TransactionsView`, and `BalancesView`. Each declares its column configuration and passes it to `DynamicTable`.

Reusable table layer (`components/ui/Table/`): `DynamicTable` accepts a column configuration and data array, and delegates to separate loading, empty, and error state components.

Data layer (`api/`): `apiClient` is a typed fetch wrapper. Domain modules (`playersApi`, `transactionsApi`, `balancesApi`) expose typed methods over it.

State layer (`hooks/useTableData.ts`): A single hook that centralizes async fetch, loading state, error state, and refetch behavior for all views.

4.3.2 Runtime Configuration Injection

In Docker, `docker-entrypoint.sh` generates `/runtime-config.js` before Nginx starts, injecting `VITE_API_URL` and `VITE_API_TIMEOUT` into the browser environment. If `VITE_API_URL` points to `localhost` or `127.0.0.1`, the entrypoint rewrites it to `/api` to prevent CORS errors.

4.4 Tech Stack

Category	Technology	Notes
Language	TypeScript 5	Strict mode enabled
Framework	React 19	
Build Tool	Vite 7	Dev server proxies `/api`
Styling	Tailwind CSS 3	
Icons	Lucide React	
Linting	ESLint 9	React Hooks + Tailwind plugins
Runtime	Nginx	Serves static build; proxies `/api/**`
Packaging	Docker multi-stage image	Node build stage + Nginx runtime

4.5 Configuration

Variable	Default	Required	Description
`VITE_API_PROXY_TARGET`	`http://localhost:3`	No	Vite dev server proxy target. Development only.
`VITE_API_URL`	—	No	Browser-visible API base URL. Rewritten to `/api` if pointing to localhost.
`VITE_API_TIMEOUT`	—	No	Request timeout in milliseconds.
`API_UPSTREAM_URL`	`http://craftalism-`	No	Nginx upstream for `/api/**` reverse proxy.

4.6 Known Limitations

- The dashboard has no authentication layer. Anyone who can reach port 8080 can view all economy data.
- "Add Player" and "Add Balance" action buttons are UI placeholders; no create flows are implemented or wired to the API.
- Sidebar navigation items are presentational only; routing between sections is not implemented.
- No automated tests exist for the API client, hooks, or table components.
- The repository workflow builds and publishes Docker images on tags but does not run linting or type checking before publication.

4.7 Roadmap

- Add OAuth2 or session-based authentication and authorization to the dashboard.
- Implement React Router and route-driven navigation for all dashboard sections.
- Build create, update, and delete flows for players, balances, and transactions.

- Add unit and integration tests for the API client, hooks, and table rendering.
- Extend the GitHub Actions workflow to run linting and type checking before publishing images.

Module 5

Craftalism Deployment

Docker Compose · PostgreSQL 18 · Nginx

Container orchestration — single-command deployment for the full platform

5.1 Overview

The Deployment repository contains the Docker Compose configuration and environment templates needed to run the entire Craftalism platform. It contains no application source code; all services are pulled from pre-built images published to the GitHub Container Registry. A single `docker compose up` command starts every service in the correct dependency order.

5.2 Responsibilities

- Defines the full service topology and inter-service networking in `docker-compose.yml`.
- Manages a one-shot database initializer that creates the `authserver` database before the Authorization Server starts.
- Downloads and installs the Craftalism Economy plugin JAR into the Minecraft server container at startup using the configured `ECONOMY_VERSION`.
- Templates an `env.example` file covering all required and optional configuration for the full stack.
- Exposes services on defined host ports for external access.

5.3 Architecture and Design

5.3.1 Service Topology

Container	Image	Purpose
`craftalism-postgres`	`postgres:18-alpine`	Primary database for API and Authorization Server.
`auth-db-init`	`postgres:18-alpine`	One-shot; creates `authserver` DB then exits.
`craftalism-auth-server`	`ghcr.io/.../authorizat`	OAuth2 token issuer.
`craftalism-api`	`ghcr.io/.../craftalism`	Core REST API.
`craftalism-dashboard`	`ghcr.io/.../craftalism`	Admin web UI.
`craftalism-minecraft`	`itzg/minecraft-server`	Paper game server + economy plugin.

5.3.2 Startup Dependency Order

1. `craftalism-postgres` starts and passes its health check.
2. `auth-db-init` runs and creates the `authserver` database, then exits.
3. `craftalism-auth-server` and `craftalism-api` start in parallel (both depend on a healthy PostgreSQL).

4. `craftalism-dashboard` and `craftalism-minecraft` start once the API is healthy.

5.3.3 Port Mapping

Service	Host Port	Notes
Dashboard	8080	HTTP. No TLS termination configured.
Craftalism API	3000	HTTP.
Authorization Server	9000	HTTP.
Minecraft	25565	Game traffic (TCP/UDP).
RCON	25575	Remote console.

5.4 Tech Stack

Category	Technology	Notes
Orchestration	Docker Compose v2	
Database	PostgreSQL 18 (alpine)	Shared instance; separate databases per service
Game Server	`itzg/minecraft-server`	Paper-compatible; plugin auto-downloaded
Networking	Docker bridge network	All services on `craftalism-network`

5.5 Configuration

Variable	Default	Required	Description
`DB_PASSWORD`	—	**Yes**	PostgreSQL password for the `craftalism` user.
`MINECRAFT_CLIENT_SECRET`	—	**Yes**	OAuth2 client secret; must match the Authorization Server.
`RSA_PRIVATE_KEY`	—	**Yes**	PEM RSA private key with literal `\n` separators.
`RSA_PUBLIC_KEY`	—	**Yes**	PEM RSA public key with literal `\n` separators.
`AUTH_ISSUER_URI`	—	**Yes**	Externally reachable URL of the Authorization Server. Must be identical across all services.
`ECONOMY_VERSION`	—	**Yes**	GitHub Release tag of the economy plugin JAR to download on Minecraft container startup.

> **Note:** See Appendix A for variables that must be consistent across multiple services, particularly `AUTH_ISSUER_URI` and `MINECRAFT_CLIENT_SECRET`.

5.6 Known Limitations

- Image tags in `docker-compose.yml` reference `latest`, making deployments non-deterministic and non-reproducible across environments.
- No TLS termination or reverse proxy is configured; all services are exposed directly on host ports over plain HTTP.
- No `generate-keys.sh` script exists despite being referenced in `env.example`. RSA keys

must be generated manually.
- No automated health verification or smoke test suite runs post-startup.

5.7 Roadmap

- Pin all image tags to specific digest or version strings for reproducible deployments.
- Add a `generate-keys.sh` script for RSA key pair generation and formatting.
- Provide an example reverse proxy configuration with TLS termination (Nginx, Caddy, or Traefik).
- Add automated post-startup smoke tests that verify all service health endpoints.

Module A

Shared Environment Variables

The following environment variables are referenced by more than one module. Each module's Configuration section documents the full set of variables for that module; this appendix highlights variables whose values must be consistent across services.

Variable	Consistency requirement
`AUTH_ISSUER_URI`	Must be set to the same value in the Craftalism API, the Authorization Server, and the Minecraft plugin. A mismatch causes JWT validation failures because the `iss` claim in issued tokens will not match what the API expects.
`MINECRAFT_CLIENT_SECRET`	Must match between the Authorization Server (where the client is seeded) and the Minecraft plugin (where it is used to request tokens).
`RSA_PRIVATE_KEY` / `RSA_PUBLIC_KEY`	Must remain stable across Authorization Server restarts. Rotating these keys invalidates all previously issued tokens.

Module B

Quick-Start Command Reference

Start the full platform

```
cd craftalism-deployment
cp env.example .env
# Edit .env with required secrets
docker compose pull
docker compose up -d
```

Health checks

```
curl -f http://localhost:3000/actuator/health   # Craftalism API
curl -f http://localhost:9000/actuator/health   # Authorization
    Server
curl -I  http://localhost:8080/                 # Dashboard
```

Run a module's tests

```
# Java modules (API, Authorization Server)
cd [module]/java && ./gradlew test

# Minecraft plugin
cd craftalism-economy/java && ./gradlew test
```

Request a token manually

```
curl -X POST 'http://localhost:9000/oauth2/token' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -u "minecraft-server:${MINECRAFT_CLIENT_SECRET}" \
  -d 'grant_type=client_credentials&scope=api:read api:write'
```